PRAKTIKUM
SISTEM BASIS DATA

Nama	**Jonathan Frederick Kosasih**	**No. Modul**	**5**
NPM	**2306225981**	**Tipe**	**CS**

1. Screenshot menjalankan query:

```
express_jonathankosasih=# CREATE TABLE IF NOT EXISTS users (
express_jonathankosasih(#  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
express_jonathankosasih(#  name VARCHAR(255) NOT NULL,
express_jonathankosasih(#  email VARCHAR(255) UNIQUE NOT NULL,
express_jonathankosasih(#  password VARCHAR(255) NOT NULL,
express_jonathankosasih(#  balance INT DEFAULT 0,
express_jonathankosasih(#  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
express_jonathankosasih(# );
CREATE TABLE
express_jonathankosasih=#
```

2. Screenshot npm install:

```
TERMINAL
PS C:\Users\jonat\OneDrive\Documents\university\smt4\sbd\prak\modul5_sbd> npm i

up to date, audited 85 packages in 2s

15 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\jonat\OneDrive\Documents\university\smt4\sbd\prak\modul5_sbd>
```

3. Source code terbaru untuk masing-masing file:

   server.js

```
const express = require('express');
require('dotenv').config();

const app = express();
const port = process.env.PORT || 3000;

app.use(express.json());

app.use("/store", require("./src/routes/store.route"));
app.use("/user", require("./src/routes/user.route"));

app.listen(port, () => {
```

```
    console.log(`Server running on port ${port}`);
});
```

store.repository.js

```
const db = require("../database/pg.database");

exports.getAllStores = async () => {
    try {
        const result = await db.query("SELECT * FROM stores");
        return result.rows;
    } catch (error) {
        console.error("Store repository error", error);
    }
};

exports.createStore = async (store) => {
    try {
        const result = await db.query(
            "INSERT INTO stores(name, address) VALUES($1, $2) RETURNING *",
            [store.name, store.address]
        );
        return result.rows[0];
    } catch (error) {
        console.error("Store repository error", error);
    }
};

exports.getStoreById = async (id) => {
    try {
         const result = await db.query("SELECT * FROM stores WHERE id = $1",
[id]);
        return result.rows[0];
    } catch (error) {
        console.error("Store repository error", error);
    }
};

exports.updateStore = async (store) => {
    try {
        const result = await db.query(
                "UPDATE stores SET name = $1, address = $2 WHERE id = $3
```

```
RETURNING *",
            [store.name, store.address, store.id]
        );
        return result.rows[0];
    } catch (error) {
        console.error("Store repository error", error);
    }
}

exports.deleteStore = async (id) => {
    try {
        const result = await db.query("DELETE FROM stores WHERE id = $1
RETURNING *", [id]);
        return result.rows[0];
    } catch (error) {
        console.error("Store repository error", error);
    }
}
```

store.controller.js

```
const storeRepository = require("../repositories/store.repository");
const baseResponse = require("../utils/baseResponse.util");

exports.getAllStores = async (req, res) => {
    try {
        const stores = await storeRepository.getAllStores();
        baseResponse(res, true, 200, "Stores found", stores);
    } catch (error) {
        baseResponse(res, false, 500, "Error retrieving stores", error);
    }
};

exports.createStore = async (req, res) => {
    if (!req.body.name || !req.body.address) {
        return baseResponse(res, false, 400, "Missing store name or address",
null);
    }
    try {
        const store = await storeRepository.createStore(req.body);
        baseResponse(res, true, 201, "Store created", store);
    }
```

```
    catch (error) {
            baseResponse(res, false, 500, error.message || "Server error",
error);
    }
};

exports.getStoreById = async (req, res) => {
    try {
        const store = await storeRepository.getStoreById(req.params.id);
        if (!store) {
            return baseResponse(res, false, 404, "Store not found", null);
        }
        baseResponse(res, true, 200, "Store found", store);
    } catch (error) {
        baseResponse(res, false, 500, "Error retrieving store", error);
    }
}

exports.updateStore = async (req, res) => {
    if (!req.body.name || !req.body.address) {
        return baseResponse(res, false, 400, "Missing store name or address",
null);
    }
    try {
        const store = await storeRepository.updateStore(req.body);
        if (!store) {
            return baseResponse(res, false, 404, "Store not found", null);
        }
        baseResponse(res, true, 200, "Store updated", store);
    }
    catch (error) {
        baseResponse(res, false, 500, "Error updating store", error);
    }
}

exports.deleteStore = async (req, res) => {
    try {
        const deleted = await storeRepository.deleteStore(req.params.id);
        if (!deleted) {
            return baseResponse(res, false, 404, "Store not found", null);
```

```
        }
        baseResponse(res, true, 200, "Store deleted", deleted);
    } catch (error) {
        baseResponse(res, false, 500, "Error deleting store", error);
    }
}
```

store.route.js

```
const storeController = require("../controllers/store.controller");
const express = require("express");
const router = express.Router();

router.get("/getAll", storeController.getAllStores);
router.post("/create", storeController.createStore);
router.get("/:id", storeController.getStoreById);
router.put("/", storeController.updateStore);
router.delete("/:id", storeController.deleteStore);
module.exports = router;
```

user.repository.js

```
const db = require("../database/pg.database");

exports.registerUser = async (user) => {
    try {
        const result = await db.query(
            "INSERT INTO users(name, email, password, balance) VALUES($1, $2,
$3, 0) RETURNING *",
            [user.name, user.email, user.password]
        );
        return result.rows[0];
    } catch (error) {
        console.error("User repository error", error);
    }
};

exports.loginUser = async (email, password) => {
    try {
         const result = await db.query("SELECT * FROM users WHERE email = $1
AND password = $2", [email, password]);
        if(result.rows.length === 0) {
            return null;
        }
```

```
        return result.rows[0];
    } catch (error) {
        console.error("User repository error", error);
    }
}

exports.getUserByEmail = async (email) => {
    try {
        const result = await db.query("SELECT * FROM users WHERE email = $1",
[email]);
        return result.rows[0];
    } catch (error) {
        console.error("User repository error", error);
    }
}

exports.updateUser = async (user) => {
    try {
        const result = await db.query(
            "UPDATE users SET name = $1, email = $2, password = $3 WHERE id =
$4 RETURNING *",
            [user.name, user.email, user.password, user.id]
        );
        return result.rows[0];
    } catch (error) {
        console.error("User repository error", error);
    }
}

exports.deleteUser = async (id) => {
    try {
            const result = await db.query("DELETE FROM users WHERE id = $1
RETURNING *", [id]);
        return result.rows[0];
    } catch (error) {
        console.error("User repository error", error);
    }
}
```

user.controller.js

```
const userRepository = require('../repositories/user.repository');
```

```
const baseResponse = require("../utils/baseResponse.util");

exports.registerUser = async (req, res) => {
    if (!req.query.name || !req.query.email || !req.query.password) {
          return baseResponse(res, false, 400, "Missing user name, email, or
password", null);
    }
    try {
        if (await userRepository.getUserByEmail(req.query.email)) {
             return baseResponse(res, false, 400, "Email already registered",
null);
        }
        const user = await userRepository.registerUser(req.query);
        baseResponse(res, true, 201, "User created", user);
    } catch (error) {
            baseResponse(res, false, 500, error.message || "Server error",
error);
    }
}

exports.loginUser = async (req, res) => {
    if (!req.query.email || !req.query.password) {
           return baseResponse(res, false, 400, "Missing email or password",
null);
    }
    try {
              const user = await userRepository.loginUser(req.query.email,
req.query.password);
        if (user === null) {
            return baseResponse(res, false, 404, "Invalid email or password",
null);
        }
        baseResponse(res, true, 200, "Login success", user);
    } catch (error) {
        baseResponse(res, false, 500, "Error retrieving user", error);
    }
}

exports.getUserByEmail = async (req, res) => {
    try {
```

```
        const user = await userRepository.getUserByEmail(req.params.email);
        if (!user) {
            return baseResponse(res, false, 404, "User not found", null);
        }
        baseResponse(res, true, 200, "User found", user);
    } catch (error) {
        baseResponse(res, false, 500, "Error retrieving user", error);
    }
}

exports.updateUser = async (req, res) => {
    if (!req.body.name || !req.body.email || !req.body.password) {
          return baseResponse(res, false, 400, "Missing user name, email, or
password", null);
    }
    try {
        const user = await userRepository.updateUser(req.body);
        if (!user) {
            return baseResponse(res, false, 404, "User not found", null);
        }
        baseResponse(res, true, 200, "User updated", user);
    } catch (error) {
        baseResponse(res, false, 500, "Error updating user", error);
    }
}

exports.deleteUser = async (req, res) => {
    try {
        const deleted = await userRepository.deleteUser(req.params.id);
        if (!deleted) {
            return baseResponse(res, false, 404, "User not found", null);
        }
        baseResponse(res, true, 200, "User deleted", deleted);
    } catch (error) {
        baseResponse(res, false, 500, "Error deleting user", error);
    }
}
```

user.route.js

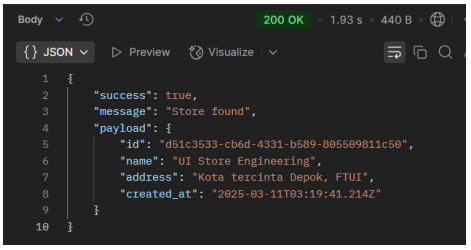
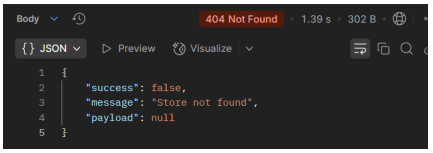
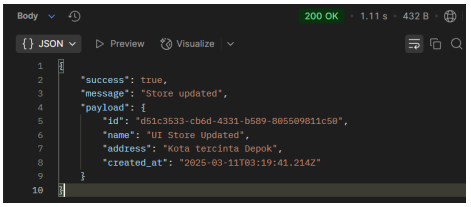
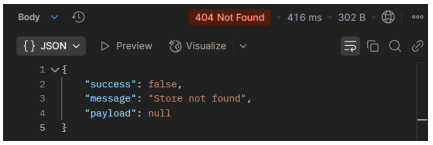
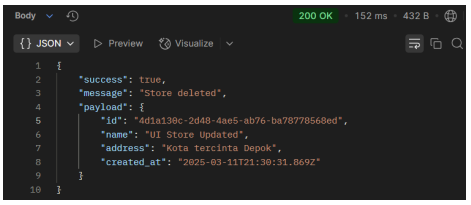
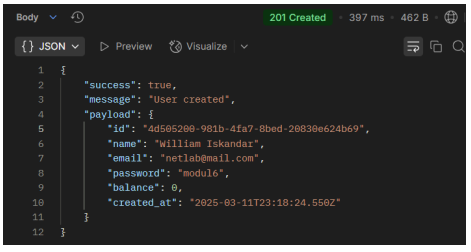
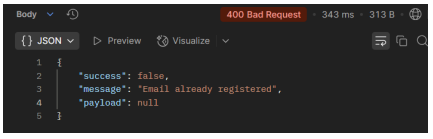
```
const userController = require('../controllers/user.controller');
const express = require('express');
```

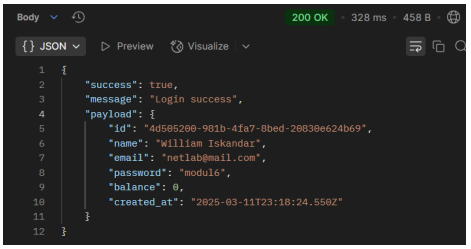
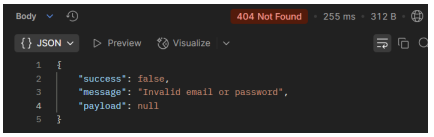
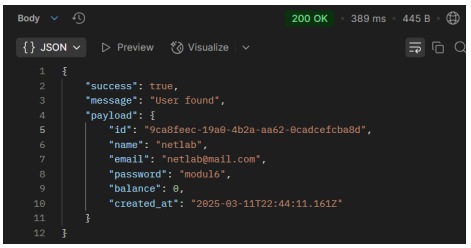
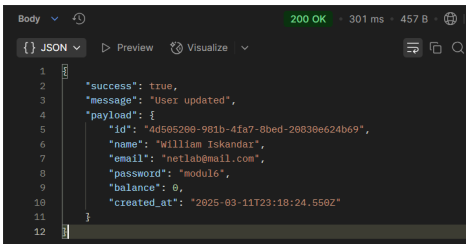
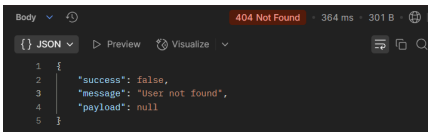
```
const router = express.Router();

router.post("/register", userController.registerUser);
router.post("/login", userController.loginUser);
router.get("/:email", userController.getUserByEmail);
router.put("/", userController.updateUser);
router.delete("/:id", userController.deleteUser);
module.exports = router;
```

*NOTE: pg.database.js, baseResponse.util.js, dan .env tidak berubah

4. Screenshot:

Method	Endpoint	Success	Failed
GET	/store/:id		
PUT	/store/		
DELETE	/store/:id		
POST	/user/register		

POST	/user/login		
GET	/user/:email		
PUT	/user		
DELETE	/user/:id